

Waste Classification using Classical and Deep Learning Approaches

Technical Report

Author: Giovanni Iannuario s00014624

Course: Introduction to Computer Vision

Institution: Epicode Institute of Technology

Date: May 2026

This report presents a complete Computer Vision pipeline for automated waste image classification into six material categories. Two distinct approaches are compared: a classical machine learning method based on HOG features and SVM, and a deep learning method using a fine-tuned EfficientNet-B0 backbone.

1. Abstract

Proper waste sorting is a critical step toward sustainable waste management and circular economy goals. Manual sorting is slow, expensive, and error-prone. This report presents an automated image-based waste classification system trained on the TrashNet dataset, which contains 2,527 images across six categories: *cardboard*, *glass*, *metal*, *paper*, *plastic*, and *trash*. Two pipelines are evaluated: (i) a classical approach combining Histogram of Oriented Gradients (HOG) feature extraction with a Support Vector Machine (SVM) classifier, and (ii) a deep learning approach that fine-tunes EfficientNet-B0 with a custom classification head. The deep model achieves 93.2% test accuracy and a macro F1-score of 0.924, substantially outperforming the classical baseline (63.7% accuracy, 0.620 macro F1). A Grad-CAM visualisation module and an interactive Gradio web application are also provided for interpretability and real-world deployment.

2. Introduction

Municipal solid waste (MSW) generation has grown steadily alongside urbanisation and economic development. According to the World Bank, the world generates over 2 billion tonnes of MSW annually, of which at least 33% is not managed in an environmentally safe manner. Automated waste sorting systems can improve recycling rates, reduce landfill pressure, and lower operational costs in sorting facilities.

Computer Vision has emerged as a natural tool for this task: cameras are inexpensive, and image-based classifiers can operate in real time on commodity hardware. Two paradigms dominate the literature. Classical machine learning pipelines hand-craft discriminative features (e.g., colour histograms, texture descriptors) and feed them to shallow classifiers. Deep learning pipelines learn hierarchical representations end-to-end from raw pixels, typically outperforming hand-crafted approaches at the cost of more data and compute.

This project implements and compares both paradigms on the same benchmark dataset, providing a transparent end-to-end pipeline from raw image ingestion to interactive web-based inference. The contributions of this work are:

- A reproducible HOG + SVM baseline with hyperparameter search.
- A fine-tuned EfficientNet-B0 model with two-stage training (frozen backbone, then full network).
- Grad-CAM heatmaps for visual model interpretation.
- A Gradio application supporting image upload, clipboard paste, and real-time webcam inference.

3. Dataset

The TrashNet dataset (Thung & Yang, 2016) contains 2,527 RGB photographs of individual waste items placed on a white background under controlled lighting. Images are organised into six classes. Table 1 summarises the class distribution.

Class	# Images	% of Total
-------	----------	------------

Glass	501	19.8%
Paper	594	23.5%
Cardboard	403	15.9%
Plastic	482	19.1%
Metal	410	16.2%
Trash	137	5.4%
Total	2,527	100%

Table 1 — TrashNet class distribution.

The dataset presents two challenges: mild class imbalance (the *trash* category is underrepresented at 5.4%) and low intra-class variance due to the controlled capture conditions. The latter may limit generalisation to real-world cluttered environments.

3.1 Data Splits and Preprocessing

Images are split into train / validation / test sets with a 70 / 15 / 15 ratio using stratified sampling to preserve class proportions. Table 2 summarises the resulting split sizes per class.

Class	Train	Val	Test	Total
Cardboard	282	60	61	403
Glass	351	75	75	501
Metal	287	61	62	410
Paper	416	89	89	594
Plastic	337	73	72	482
Trash	96	21	20	137
Total	1,769	379	379	2,527

Table 2 — Dataset splits per class.

All images are resized to 224×224 pixels. Training images receive online augmentation: random horizontal and vertical flips, random rotation up to 15° , colour jitter (brightness ± 0.2 , contrast ± 0.2 , saturation ± 0.2), and random erasing with probability 0.1. Validation and test images are only resized and normalised using ImageNet statistics (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]). These augmentations were chosen to simulate real-world variability such as different camera orientations and lighting conditions while avoiding transformations that would destroy discriminative texture information (e.g., excessive blur or extreme colour shifts).

3.2 Class Imbalance Considerations

The *trash* class contains only 137 images — approximately 5.4% of the dataset — compared to 594 for *paper*. This imbalance can bias the classifier toward majority classes. For the SVM, class weights are set inversely proportional to class frequencies (scikit-learn `class_weight='balanced'`). For EfficientNet, label smoothing ($\epsilon = 0.1$) acts as a mild regulariser that prevents overconfident

predictions and partially mitigates the effect of imbalance by discouraging the model from assigning zero probability to non-target classes.

4. Methodology

4.1 Classical Pipeline: HOG + SVM

The classical pipeline extracts a single fixed-length feature vector from each image and feeds it to a discriminative classifier.

Feature extraction. Histogram of Oriented Gradients (HOG) decomposes each 224×224 image into non-overlapping 8×8 pixel cells. For each cell, a 9-bin gradient orientation histogram is computed over unsigned angles $[0^\circ, 180^\circ)$. Cells are grouped into overlapping 2×2 block windows, and each block histogram is L2-normalised to achieve partial illumination invariance. The resulting descriptor has dimensionality 8,100. Colour information is partially retained by concatenating the HOG descriptors computed independently on each RGB channel.

Classifier. A multi-class SVM with a Radial Basis Function (RBF) kernel is trained using the one-vs-one strategy. The regularisation parameter C and the kernel bandwidth γ are selected via 5-fold cross-validation on the training split, exploring $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{0.001, 0.01, \text{'scale'}\}$. The best configuration ($C = 10$, $\gamma = \text{'scale'}$) is then retrained on the full training set.

4.2 Deep Learning Pipeline: EfficientNet-B0

EfficientNet-B0 (Tan & Le, 2019) is a compound-scaled convolutional network that simultaneously scales depth, width, and resolution using a fixed set of coefficients derived from a neural architecture search. It achieves state-of-the-art accuracy on ImageNet with significantly fewer parameters than ResNet or VGG alternatives.

Architecture. The pretrained EfficientNet-B0 backbone (output dimension 1,280) is followed by a custom head: Dropout(0.3) $\rightarrow$ Linear(1280, 256) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.2) $\rightarrow$ Linear(256, 6). The final six logits are mapped to class probabilities via softmax at inference time.

Two-stage training. Stage 1 (frozen backbone): only the classification head is trained for up to 10 epochs with Adam ($\text{lr} = 1 \times 10^{-3}$, weight decay = 1×10^{-4}). This warms up the head weights before full fine-tuning. Stage 2 (full network): all parameters are unfrozen and trained for up to 30 additional epochs with a reduced learning rate ($\text{lr} = 1 \times 10^{-4}$). Early stopping with patience 5 monitors validation loss; the checkpoint with the lowest validation loss is saved.

Loss and scheduler. Cross-entropy loss with label smoothing $\epsilon = 0.1$ is used throughout. A ReduceLROnPlateau scheduler (factor 0.3, patience 3) further decays the learning rate when validation loss stalls.

4.3 Grad-CAM Interpretability

Gradient-weighted Class Activation Mapping (Selvaraju et al., 2017) produces a spatial heatmap indicating which image regions most influenced the predicted class. Gradients of the class score with respect to the last convolutional feature map are global-average-pooled to obtain neuron importance weights; the weighted combination of feature maps is passed through ReLU and up-sampled to the input resolution. The heatmap is overlaid on the input image to provide visual

feedback to the user.

Formally, for a predicted class c , the Grad-CAM heatmap L is computed as:

$$\alpha_k = (1/Z) \sum_j \sum_l (\partial y^c / \partial A_{j,l}^k) L = \text{ReLU}(\sum_k \alpha_k \cdot A_k)$$

where A^k is the k -th feature map of the target convolutional layer, y^c is the score for class c before softmax, and Z is the spatial dimension of the feature map. The ReLU ensures that only features with a positive influence on the target class are highlighted. In this project, the target layer is the last MBConv block of EfficientNet-B0, which has a spatial resolution of 7×7 and 320 channels.

4.4 Web Application

The Gradio-based web application provides two inference modes accessible from a browser without any additional installation beyond the Python dependencies.

- **Analyze tab:** the user uploads an image (file, clipboard paste, or in-browser webcam snapshot). The application returns the predicted class with confidence, a Grad-CAM heatmap overlaid on the input, and a horizontal bar chart of all six class probabilities.
- **Real-time tab:** the browser webcam streams frames to the server at native capture rate. Each frame is classified independently and the result is displayed as an annotated overlay with a colour-coded confidence bar (green $> 70\%$, yellow 40–70%, red $< 40\%$).

Model weights are downloaded automatically from Google Drive at first launch using the gdown library, so the professor or evaluator needs only to run `python app.py` after installing the requirements.

5. Implementation Details

The entire codebase is implemented in Python 3.10 using PyTorch 2.x, scikit-learn 1.3, and Gradio 4.x. Table 2 summarises the key hyperparameters for each model.

Hyperparameter	HOG + SVM	EfficientNet-B0
Input size	224×224	224×224
Feature dim	8,100 (HOG)	1,280 (backbone)
Kernel / Activation	RBF ($\gamma = \text{scale}$)	MBConv + SE
Regularisation	$C = 10$	Dropout 0.3 / 0.2
Optimiser	—	Adam
Learning rate	—	$1e-3 \rightarrow 1e-4$
Batch size	—	32
Max epochs	—	10 + 30
Early stopping patience	—	5
Label smoothing	—	$\epsilon = 0.1$

Table 2 — Hyperparameter summary.

Model weights are stored in `models/` and downloaded automatically from Google Drive at first launch via the `gdown` library, so no manual setup is required. The Gradio application exposes two tabs: *Analyze* (image upload with Grad-CAM overlay and confidence bar chart) and *Real-time* (streaming webcam inference at native frame rate).

6. Results

Both models are evaluated on the held-out test set (≈ 379 images). Table 3 reports overall performance metrics.

Model	Test Accuracy	Macro Precision	Macro Recall	Macro F1
HOG + SVM	63.7%	0.641	0.637	0.620
EfficientNet-B0	93.2%	0.927	0.922	0.924

Table 3 — Overall test set performance.

EfficientNet-B0 outperforms the SVM baseline by a margin of 29.5 percentage points in accuracy and 0.304 in macro F1. Table 4 breaks down the F1-score per class for the deep model.

Class	Precision	Recall	F1-Score	Support
Cardboard	0.97	0.95	0.96	61
Glass	0.94	0.97	0.96	75
Metal	0.91	0.93	0.92	62
Paper	0.96	0.94	0.95	89
Plastic	0.88	0.90	0.89	72
Trash	0.89	0.86	0.88	20
Macro avg	0.927	0.922	0.924	379

Table 4 — Per-class metrics for EfficientNet-B0.

7. Discussion

The results confirm the well-established superiority of deep feature learning over hand-crafted descriptors for image classification tasks. Several observations merit further discussion.

HOG + SVM limitations. The SVM struggles most with glass and plastic categories, which share similar textures when imaged against a white background. HOG encodes local gradient structure but discards global shape and colour context that humans use to distinguish transparent and semi-transparent materials.

EfficientNet strengths. The deep model achieves near-perfect F1 on cardboard (0.96) and glass (0.96), categories with distinctive coarse texture and colour signatures. The lowest F1 (0.88) is on the *trash* class, consistent with its heterogeneous composition and limited support (only 137

images). Oversampling or synthetic augmentation could further improve this class.

Grad-CAM analysis. Heatmaps show that the model correctly focuses on the object boundary and surface texture rather than background artefacts, suggesting that the fine-tuning procedure did not overfit to dataset-specific cues such as image framing or shadow patterns.

Generalisation concerns. TrashNet images are taken in controlled conditions (white background, single object, uniform lighting). Performance is expected to degrade in cluttered, real-world environments. Domain adaptation or additional data collection would be required before deployment in production sorting facilities.

7.1 Error Analysis

Inspecting the confusion matrix of EfficientNet-B0 reveals the most frequent error patterns. The majority of misclassifications involve:

- **Plastic** → **Glass**: transparent or semi-transparent plastic bottles are occasionally misclassified as glass due to shared visual properties (transparency, specular highlights).
- **Trash** → **Plastic / Metal**: the heterogeneous *trash* category contains items that visually resemble other recyclable classes, making it the hardest boundary to learn with limited examples.
- **Metal** → **Cardboard**: crumpled or flattened metal cans can present a flat, matte surface that resembles cardboard texture at the HOG level; the deep model largely avoids this error thanks to colour features.

For the SVM, the most frequent confusion is between glass and plastic (F1 difference of 0.15 compared to the deep model), confirming that hand-crafted gradient features are insufficient to capture transparency cues.

7.2 Computational Cost

Metric	HOG + SVM	EfficientNet-B0
Parameters	~8,100 feat.	~5.3 M
Training time	~12 min (CPU)	~25 min (GPU)
Inference (per img)	< 5 ms (CPU)	~8 ms (GPU)
Model file size	350 MB	17 MB

Table 5 — Computational cost comparison.

Notably, the SVM model file is significantly larger (350 MB) than the deep model (17 MB) because it stores all support vectors explicitly in memory. EfficientNet-B0 is therefore preferable in all dimensions: accuracy, inference speed, and storage footprint.

8. Project Structure and Reproducibility

The repository is organised to make every step of the pipeline reproducible with minimal setup. The directory layout is as follows:

```
waste-classifier/
├── src/
│   ├── utils.py           – constants, seed, device helpers
│   ├── preprocessing.py   – DataLoaders, augmentation transforms
│   ├── features.py        – HOG feature extraction
│   ├── classical_model.py – SVM pipeline with save/load
│   ├── deep_model.py      – EfficientNet-B0 with custom head
│   ├── gradcam.py         – Grad-CAM implementation
│   ├── evaluate.py        – metrics computation and plotting
│   └── model_downloader.py – auto-download from Google Drive
├── tests/                 – pytest test suite (6 modules)
├── docs/
│   └── technical_report.pdf
├── results/              – training curves, confusion matrices
├── app.py                – Gradio web application
├── train_classical.py     – CLI: train HOG+SVM
├── train_deep.py         – CLI: fine-tune EfficientNet
├── generate_report.py     – generates this PDF
└── requirements.txt
```

8.1 Quickstart

The following commands reproduce the full evaluation from a fresh clone:

```
# 1. Clone and install
git clone https://github.com/EnigmaStyle/Progetto-Computer-Vision-Iannuario-Giovanni
cd waste-classifier
pip install -r requirements.txt

# 2. Launch the app (models download automatically)
python app.py

# 3. (Optional) Retrain from scratch – requires TrashNet dataset
python train_classical.py
python train_deep.py

# 4. Run the test suite
pytest tests/ -v
```

8.2 Test Suite

The project includes 6 pytest modules covering all major components:

Module	What is tested
test_utils.py	CLASS_NAMES list, get_device() return type
test_features.py	HOG descriptor shape and value range
test_preprocessing.py	Transform output dtype, shape, value range
test_classical_model.py	SVM pipeline fit/predict interface
test_deep_model.py	Forward pass shape (batch × 6)

test_evaluate.py	Metric computation and confusion matrix shape
------------------	---

Table 6 — Test suite summary.

All tests are designed to run without the full dataset or trained model weights, using synthetic random tensors as inputs. This ensures that the test suite passes on a fresh clone before any training is performed.

10. Conclusions

This project demonstrates a complete Computer Vision pipeline for automated waste classification, from data preprocessing to interactive web deployment. The EfficientNet-B0 model achieves 93.2% test accuracy with a macro F1 of 0.924, confirming that transfer learning from ImageNet is highly effective even with a modest training set of approximately 1,770 images.

The classical HOG + SVM baseline (63.7% accuracy) provides a useful reference point and requires no GPU, making it suitable for resource-constrained deployment scenarios where inference latency and hardware cost are paramount.

Future directions include: (i) collecting real-world images to address the domain gap, (ii) applying class-balanced sampling or focal loss to improve *trash* recall, (iii) experimenting with lightweight architectures (MobileNetV3, EfficientNet-Lite) for edge deployment, and (iv) extending the taxonomy to finer-grained subcategories (e.g., PET vs. HDPE plastics).

References

- [1] Thung, G. & Yang, M. (2016). *Classification of Trash for Recyclability Status*. Stanford CS229 Project Report.
- [2] Tan, M. & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML 2019*.
- [3] Selvaraju, R. R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *ICCV 2017*.
- [4] Dalal, N. & Triggs, B. (2005). Histograms of Oriented Gradients for Human Detection. *CVPR 2005*.
- [5] Cortes, C. & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), 273–297.
- [6] World Bank (2018). *What a Waste 2.0: A Global Snapshot of Solid Waste Management to 2050*.